

Laboratorio bases de datos primer corte: Javascript Avanzado

Laura Vanessa Barrero, Gustavo Andres Ceballos

Facultad de Ingeniería

Universidad San Buenaventura

Bogotá, Colombia

Correo: lvbarrera@academia.usbbog.edu.co, Gaceballosr@academia.usbbog.edu.co

Resumen—En este laboratorio se analiza el modelo de ejecución asincrónica de JavaScript mediante un simulador de ocho eventos temporizados, implementado con callbacks anidados, promesas encadenadas y `async/await`. Para cada evento se registran el tiempo programado y el tiempo real de ejecución, permitiendo observar las desviaciones producidas durante el procesamiento de las tareas. Se plantea un caso límite que combina temporizadores, una microtarea basada en una promesa y el bloqueo intencional de la pila de ejecución. Este escenario permite evidenciar cómo una operación síncrona prolongada retrasa los temporizadores y cómo las microtareas son atendidas antes de continuar con las tareas pendientes

Index Terms—Event Loop, Call Stack, Macrotareas, Microtareas, JavaScript, Asincronismo, Métodos Funcionales.

I. INTRODUCCIÓN

El comportamiento asincrónico de JavaScript permite programar operaciones que no necesariamente se ejecutan en el mismo instante en que son solicitadas. Para comprender este proceso resulta necesario considerar la interacción entre la pila de ejecución (Call Stack), el Event Loop, las tareas temporizadas y las microtareas asociadas a las promesas. A partir de este modelo, JavaScript ofrece diferentes formas de organizar operaciones asincrónicas, entre ellas los callbacks, las promesas y la sintaxis `async/await`. Aunque las tres alternativas pueden producir resultados equivalentes, presentan diferencias importantes en la forma de expresar el orden de ejecución, la legibilidad y el manejo de errores. Por ello, este laboratorio compara estas implementaciones a partir de un mismo conjunto de eventos, sus tiempos reales de ejecución y un caso límite diseñado para observar el comportamiento de las colas de tareas cuando la pila permanece ocupada.

II. COMPARACIÓN TÉCNICA ENTRE CALLBACKS, PROMESAS ENCADENADAS Y ASYNC

II-A. Callbacks Anidados (Callback Hell)

La aproximación clásica pasa funciones como argumentos que son invocadas tras la resolución de una tarea asíncrona.

¿Qué es? Un callback es una función que se pasa como parametro a otra función para que sea llamada cuando termine una tarea.

¿Cómo funciona en el código? Para lograr que los eventos con los que estamos trabajando corran en la secuencia correcta debemos poner cada llamada dentro de la función anterior. Aquí cada evento empieza después de terminar el que le

precede. Como el laboratorio nos pide callbacks ANIDADOS, ponemos cada llamada dentro del callback de la anterior lo cual nos crea la estructura conocida como Callback Hell (estructura tipo piramide). Cuando termina el último evento, se analiza el arreglo.

Ventajas: Funciona para operaciones simples de un solo nivel. No necesita abstraer objetos de estado futuro. Funciona muy bien en entornos antiguos.

Desventajas: Provoca pirámides información (*Pyramid of Doom*), dificultando la lectura. El manejo de errores requiere verificar manualmente la presencia de excepciones en cada nivel, lo cual hace difícil rastrear la información.

```
import { baseEvents } from "../datos.js";
import { createContext, waitForCallback, processResults }
from "../funciones.js";

const context = createContext();

console.log("=== CALLBACKS ANIDADOS ===");

/*
 * Como el laboratorio pide callbacks ANIDADOS, ponemos
 * cada llamada dentro del callback de la anterior.
 */
waitForCallback(baseEvents[0], context, function () {
  waitForCallback(baseEvents[1], context, function () {
    waitForCallback(baseEvents[2], context, function () {
      waitForCallback(baseEvents[3], context, function () {
        waitForCallback(baseEvents[4], context, function () {
          waitForCallback(baseEvents[5], context, function () {
            waitForCallback(baseEvents[6], context, function () {
              waitForCallback(baseEvents[7], context, function () {
                // Cuando terminó el último evento, analizamos
                // el arreglo.
                processResults(context.results);
              });
            });
          });
        });
      });
    });
  });
});
```

Figura 1. Código callbacks anidados

II-B. Promesas Encadenadas (`.then`)

Una Promise representa una operación que terminará en el futuro.

`.then()` indica qué hacer cuando la Promise termina correctamente. `.catch()` permite manejar un error de la cadena.

Al igual que sucede con los callbacks un evento comienza después de terminar el anterior. La diferencia principal es que aquí el orden lo vemos como una cadena de `.then()`.

Ventajas: Convierte el anidamiento horizontal en flujo vertical paso a paso lo cual nos brinda una legibilidad mucho más cómoda. Permite la propagación centralizada al final de la cadena convirtiéndolo en un unico bloque con `.catch()`, tenemos cola de microtarear

Desventajas: La syntaxs repetitiva nos puede causar problemas ya que retorna funciones anónimas implícitas en cada etapa, lo que puede complicar la escritura si los contextos que estamos trabajando requieren compartir variables.

```
import { baseEvents } from "../datos.js";
import { createContext, waitForPromise,
processResults } from "../funciones.js";

const context = createContext();

console.log("=== PROMESAS ENCADENADAS ===");

/*
La idea es la misma que en callbacks: un evento comienza
de terminar el anterior.

La diferencia es que aquí el orden se ve como una cadena
*/

waitForPromise(baseEvents[0], context)
  .then(function () {
    return waitForPromise(baseEvents[1], context);
  })
  .then(function () {
    return waitForPromise(baseEvents[2], context);
  })
  .then(function () {
    return waitForPromise(baseEvents[3], context);
  })
  .then(function () {
    return waitForPromise(baseEvents[4], context);
  })
  .then(function () {
    return waitForPromise(baseEvents[5], context);
  })
  .then(function () {
    return waitForPromise(baseEvents[6], context);
  })
  .then(function () {
    return waitForPromise(baseEvents[7], context);
  })
  .then(function () {
    processResults(context.results);
  })
  .catch(function (error) {
    console.log("Ocurrió un error: " + error.message);
  });
```

Figura 2. Código promesas encadenadas

II-C. Sintaxis `async/await`

Azúcar sintáctica sobre las promesas que permite escribir código asíncronico con apariencia síncrona. Async permite crear una función asíncronica. `await` espera a que una Promise

termine antes de continuar con la siguiente línea de ESTA función. No bloquea todo JavaScript; solo pausa esta función `async`.

Ventajas: Nos da una Legibilidad mucho más óptima en comparación a nuestras anteriores opciones ya que se lee de manera sencilla de arriba a abajo. Integración nativa con estructuras de control estándar como bloques `try/catch` lo cual nos da un manejo de errores nativo, facilita la búsqueda de información dentro del código.

Desventajas: Riesgo de bloquear ejecuciones paralelas si se utiliza `await`, se puede caer en la pausa innecesaria de tareas que podrían ser paralelas.

```
import { baseEvents } from "../datos.js";
import { createContext, waitForPromise, processResults } from "../funciones.js";

const context = createContext();

console.log("=== ASYNC / AWAIT ===");

async function execute() {
  try {
    // Se lee de arriba hacia abajo.
    await waitForPromise(baseEvents[0], context);
    await waitForPromise(baseEvents[1], context);
    await waitForPromise(baseEvents[2], context);
    await waitForPromise(baseEvents[3], context);
    await waitForPromise(baseEvents[4], context);
    await waitForPromise(baseEvents[5], context);
    await waitForPromise(baseEvents[6], context);
    await waitForPromise(baseEvents[7], context);

    // Cuando terminaron los ocho eventos, procesa
    processResults(context.results);
  } catch (error) {
    // try/catch permite manejar los errores.
    console.log("Ocurrió un error: " + error.message);
  }
}

// Llamamos la función para iniciar el programa.
execute();
```

Figura 3. Código `async`

II-D. Recomendación Técnica para Backend

Según los resultados y aprendizajes que hemos obtenido a lo largo de este laboratorio hemos notado que para el desarrollo de backend con Node.js es mucho mas facil y recomendable usar la estructura de **`async/await`** junto con **`promises`** ya que esto nos garantiza una lectura más limpia, un código mas ordenado y un mayor control sobre los posibles errores.

III. PREDICCIÓN Y RECONCILIACIÓN DEL EVENT LOOP

III-A. Predicción Teórica

Si los eventos se ejecutaran exactamente en el tiempo en que fueron programados, el orden de salida en la consola debería ser de menor a mayor según su tiempo de espera:

EV-05 (60 ms) → EV-03 (80 ms) → EV-07 (90 ms) →
 EV-01 (100 ms) → EV-06 (110 ms) → EV-02 (130 ms) →
 EV-08 (140 ms) → EV-04 (160 ms)

III-B. Mecanismo Interno y Reconciliación

Al usar `setTimeout`, Node.js envía la tarea a las Web APIs / Node APIs y pone a correr un temporizador de fondo. Cuando la espera termina, el callback no se ejecuta de inmediato: pasa a la cola de macrotareas (*Task Queue*).

El **Event Loop** solo toma ese callback y lo pasa a la pila de ejecución (*Call Stack*) si la pila se encuentra totalmente libre. Por esta razón, el tiempo real siempre resulta un poco mayor al tiempo programado ($\Delta t = t_{\text{real}} - t_{\text{programado}}$). Esa pequeña diferencia de milisegundos (latencia) ocurre porque la pila estaba ocupada procesando otra tarea síncrona o por la sobrecarga propia del sistema operativo.

IV. PROCESAMIENTO FUNCIONAL DEL ARREGLO

Cálculo de Latencia Promedio con `reduce`: Recorre los resultados y acumula la diferencia de cada respuesta para obtener el total de la desviación y dividirlo entre la cantidad de eventos.

Filtrado e Identificación con `filter` Crea un nuevo arreglo que contiene los objetos los cuales su desfase o latencia supera el valor de `THRESHOLD` (5 ms).

Filtrado e Identificación con `map` Toma el arreglo resultante de `filter` y transforma el objeto extrayendo su propiedad `.id`.

Búsqueda del Primer Evento Fuera de Orden con `find` Compara el orden en el que se registraron los eventos contra una copia del arreglo ordenada por su tiempo programado (`scheduledTime`). `find` se detiene cuando detecta el primer elemento que no coincide con lo esperado.

Justificación: Tenemos un arreglo con objetos de la forma:

$$\mathcal{E} = \{\text{id}, \text{tipo}, \text{tiempoProgramado}, \text{tiempoReal}\}$$

Se buscarán métodos purificados de arreglos sin mutaciones de estado.

IV-A. Latencia Promedio con `reduce`

Se define la desviación como $d_i = |t_{\text{real},i} - t_{\text{programado},i}|$. La latencia promedio $\bar{L}$ representa el retraso sistemático medio impuesto por el Event Loop:

$$\bar{L} = \frac{1}{N} \sum_{i=1}^N |t_{\text{real},i} - t_{\text{programado},i}| \quad (1)$$

IV-B. Filtro de Desviaciones con `filter` y `map`

Se utilizó `filter` para aislar eventos cuyo $d_i > \text{UMBRAL}$ (5 ms), seguido de `map` para proyectar el resultado únicamente a los identificadores `id`.

IV-C. Detección Fuera de Orden con `find`

Se comparó el arreglo de ejecución real contra una copia ordenada mediante la función de criterio $t_{\text{programado}}$. `find` lo cual hace que retorne el primer elemento cuya posición difería del orden ideal esperado.

IV-D. Justificación Técnica de Selección de Métodos

- **map vs forEach:** `map` garantiza inmutabilidad devolviendo una nueva proyección sin efectos secundarios. `forEach` carece de retorno (*undefined*) y fomenta mutaciones en variables externas.
- **reduce vs Bucles Síncronos:** `reduce` pliega colecciones a valores escalares mediante un acumulador puro, eliminando variables contadoras externas.

Cuadro I
JUSTIFICACIÓN TÉCNICA: PROGRAMACIÓN FUNCIONAL VS. BUCLES E INMUTABILIDAD

Criterio	Bucles Tradicionales (<code>for</code> , <code>while</code>) / <code>forEach</code>	Métodos Funcionales (<code>map</code> , <code>filter</code> , <code>reduce</code> , <code>find</code>)
Mutabilidad	Modifican variables externas (acumuladores o arreglos globales) durante la iteración.	Inmutabilidad: No modifican la estructura del arreglo original; retornan estructuras o valores totalmente nuevos.
Retorno de Valor	Los bucles no devuelven ningún valor. <code>forEach</code> siempre retorna <i>undefined</i> .	Cada método funcional retorna explícitamente lo que se necesita: un valor transformado (<code>map</code>), un subconjunto (<code>filter</code>), un acumulado (<code>reduce</code>) o un único elemento (<code>find</code>).
Efecto Colateral (Side Effects)	Dependen de modificar el estado global del programa para registrar resultados.	Funciones Puras: Evitan efectos colaterales al operar solo con los datos de entrada y retornar un resultado sin alterar el entorno.

```
> node src/callbacks.js

=== CALLBACKS ANIDADOS ===
EV-01 | aviso corto | programado=100 ms |
      real=115 ms | latencia=15 ms
EV-02 | aviso medio | programado=130 ms |
      real=254 ms | latencia=124 ms
EV-03 | aviso corto | programado=80 ms |
      real=349 ms | latencia=269 ms
EV-04 | aviso largo | programado=160 ms |
      real=520 ms | latencia=360 ms
EV-05 | aviso corto | programado=60 ms |
      real=584 ms | latencia=524 ms
EV-06 | aviso medio | programado=110 ms |
      real=708 ms | latencia=598 ms
EV-07 | aviso corto | programado=90 ms |
      real=802 ms | latencia=712 ms
EV-08 | aviso largo | programado=140 ms |
      real=943 ms | latencia=803 ms

PROCESAMIENTO FUNCIONAL
Latencia promedio: 425.63 ms
Eventos con latencia mayor a 5 ms: EV-01,
EV-02, EV-03, EV-04, EV-05, EV-06, EV-07,
EV-08
Primer evento fuera de orden: EV-01
```

Figura 4. Resultados: callbacks

```

> node src/promesas.js

=== PROMESAS ENCADENADAS ===
EV-01 | aviso corto | programado=100 ms | real=111 ms | latencia=11 ms
EV-02 | aviso medio | programado=130 ms | real=251 ms | latencia=121 ms
EV-03 | aviso corto | programado=80 ms | real=342 ms | latencia=262 ms
EV-04 | aviso largo | programado=160 ms | real=513 ms | latencia=353 ms
EV-05 | aviso corto | programado=60 ms | real=575 ms | latencia=515 ms
EV-06 | aviso medio | programado=110 ms | real=699 ms | latencia=589 ms
EV-07 | aviso corto | programado=90 ms | real=793 ms | latencia=703 ms
EV-08 | aviso largo | programado=140 ms | real=936 ms | latencia=796 ms

PROCESAMIENTO FUNCIONAL
Latencia promedio: 418.75 ms
Eventos con latencia mayor a 5 ms: EV-01, EV-02, EV-03, EV-04, EV-05, EV-06, EV-07, EV-08
Primer evento fuera de orden: EV-01

```

Figura 5. Resultados: promesas encadenadas

```

=== ASYNC / AWAIT ===
EV-01 | aviso corto | programado=100 ms | real=121 ms | latencia=21 ms
EV-02 | aviso medio | programado=130 ms | real=261 ms | latencia=131 ms
EV-03 | aviso corto | programado=80 ms | real=353 ms | latencia=273 ms
EV-04 | aviso largo | programado=160 ms | real=525 ms | latencia=365 ms
EV-05 | aviso corto | programado=60 ms | real=602 ms | latencia=542 ms
EV-06 | aviso medio | programado=110 ms | real=728 ms | latencia=618 ms
EV-07 | aviso corto | programado=90 ms | real=822 ms | latencia=732 ms
EV-08 | aviso largo | programado=140 ms | real=978 ms | latencia=838 ms

PROCESAMIENTO FUNCIONAL
Latencia promedio: 440.00 ms
Eventos con latencia mayor a 5 ms: EV-01, EV-02, EV-03, EV-04, EV-05, EV-06, EV-07, EV-08
Primer evento fuera de orden: EV-01

```

Figura 6. Resultados: ASYNC/AWAIT

V. DISEÑO Y EXPLICACIÓN DEL CASO LÍMITE

V-A. Escenario Propuesto

Se registran varios temporizadores y después vamos a bloquear voluntariamente la pila de ejecución durante aproximadamente 120 ms.

También creamos una microtarea con `Promise.resolve().then()`.

¿Qué queremos demostrar? Aunque los timers ya hayan terminado su tiempo de espera, NO pueden ejecutar sus callbacks mientras la pila de ejecución siga ocupada.

Cuando la pila finalmente queda libre: 1. primero se procesa la microtarea pendiente; 2. después pueden ejecutarse los callbacks de los timers.

Este caso sirve para observar call stack, microtareas y macrotareas sin usar un ejemplo de timestamp idéntico, una excepción en `.then()` ni una Promise que nunca resuelve.

V-B. PREDICCIÓN DEL CASO

1. Se registran los timers en Web APIs / Node.
2. Se registra la microtarea en la cola de microtareas.
3. `BlockStack()` ocupa el Call Stack por 120 ms.
4. Durante ese tiempo vencen los cuatro timers y se mueven a la cola de macrotareas.
5. Al quedar libre el Call Stack, el Event Loop atiende PRIMERO la cola de microtareas.
6. Después se ejecutan las macrotareas (los callbacks de los timers).
7. Las latencias serán elevadas (¿100 ms) ya que todos los callbacks sufrieron el retraso impuesto por el bloqueo del Call Stack.

```
import { processResults } from "../funciones.js";

// Creamos eventos especiales para este caso.

const limitEvents = [
  { id: "CL-01", type: "timer", wait: 20 },
  { id: "CL-02", type: "timer", wait: 40 },
  { id: "CL-03", type: "timer", wait: 60 },
  { id: "CL-04", type: "timer", wait: 80 }
];

const startTime = Date.now();
const results = [];

console.log("=== CASO LÍMITE: PILA BLOQUEADA ===");

/*
  Esta función ocupa la pila de ejecución durante el
  El while se usa A PROPÓSITO: necesitamos impedir que
  pueda atender otras tareas durante unos milisegundos
*/

function blockStack(milliseconds) {
  const startBlock = Date.now();

  while (Date.now() - startBlock < milliseconds) {
    // No hacemos nada. Solo mantenemos ocupada la pila
  }
}

/*
  Registramos los cuatro timers.
  forEach se usa para recorrer el arreglo; NO es el
  funcional solicitado en el Paso 4.
*/

limitEvents.forEach(function (definition) {
  const scheduledTime = definition.wait;

  setTimeout(function () {
    const realTime = Date.now() - startTime;

```

Figura 7. Diseño caso límite, parte 1

```
limitEvents.forEach(function (definition) {
  setTimeout(function () {
    const eventObj = {
      scheduledTime: scheduledTime,
      realTime: realTime
    };

    results.push(eventObj);

    const delay = realTime - scheduledTime;

    console.log(
      eventObj.id + " | programado=" +
      eventObj.scheduledTime + " ms | real=" +
      eventObj.realTime + " ms | latencia=" +
      delay + " ms"
    );

    // Cuando ya se guardaron los 4 timers, procesamos
    if (results.length === limitEvents.length) {
      processResults(results);
    }
  }, definition.wait);
});

/*
  Creamos una microtarea.
  Su mensaje NO aparece inmediatamente porque todavía
  ejecutando el código principal.
*/

Promise.resolve().then(function () {
  console.log("\nMICROTAREA: se ejecutó antes de los");
});

console.log("La pila se bloqueará durante 120 ms...");

// Mientras esta función se ejecuta, los timers no pueden
blockStack(120);

console.log("La pila quedó libre.");
```

Figura 9. Diseño caso límite, parte 3

```
limitEvents.forEach(function (definition) {
  const scheduledTime = definition.wait;

  setTimeout(function () {
    const realTime = Date.now() - startTime;

    // Objeto con la estructura estricta exigida
    const eventObj = {
      id: definition.id,
      type: definition.type,
      scheduledTime: scheduledTime,
      realTime: realTime
    };

    results.push(eventObj);

    const delay = realTime - scheduledTime;

    console.log(
      eventObj.id + " | programado=" +
      eventObj.scheduledTime + " ms | real=" +
      eventObj.realTime + " ms | latencia=" +
      delay + " ms"
    );

    // Cuando ya se guardaron los 4 timers, procesamos
    if (results.length === limitEvents.length) {
      processResults(results);
    }
  }, definition.wait);
});

/*
  Creamos una microtarea.
  Su mensaje NO aparece inmediatamente porque todavía
  ejecutando el código principal.
*/
```

Figura 8. Diseño caso límite, parte 2

```
> node src/caso-limite.js

=== CASO LÍMITE: PILA BLOQUEADA ===
La pila se bloqueará durante 120 ms...
La pila quedó libre.

MICROTAREA: se ejecutó antes de los timers pendientes.
CL-01 | programado=20 ms | real=126 ms | latencia=106 ms
CL-02 | programado=40 ms | real=126 ms | latencia=86 ms
CL-03 | programado=60 ms | real=126 ms | latencia=66 ms
CL-04 | programado=80 ms | real=126 ms | latencia=46 ms

PROCESAMIENTO FUNCIONAL
Latencia promedio: 76.00 ms
Eventos con latencia mayor a 5 ms: CL-01, CL-02, CL-03, CL-04
Primer evento fuera de orden: ninguno
```

Figura 10. Resultados del caso límite

V-C. Resultado Observado y Causabilidad

1. Se inicializan los temporizadores de 20, 40, 60 y 80 ms.
2. Se inserta la microtarea a la *Microtask Queue*.
3. La función *bloquearPila(120)* congela el *Call Stack* durante 120ms.
4. Transcurridos los 120ms, los 4 temporizadores han expirado y aguardan en la *Callback Queue* (Macrotareas).

5. Al vaciarse el *Call Stack*, el Event Loop procesa ****primero la totalidad de la Cola de Microtarear**, imprimiendo el mensaje de la promesa.
6. Posteriormente, el Event Loop procesa las macrotarear pendientes en la cola. La latencia de todos los eventos registra valores superiores a 100ms.

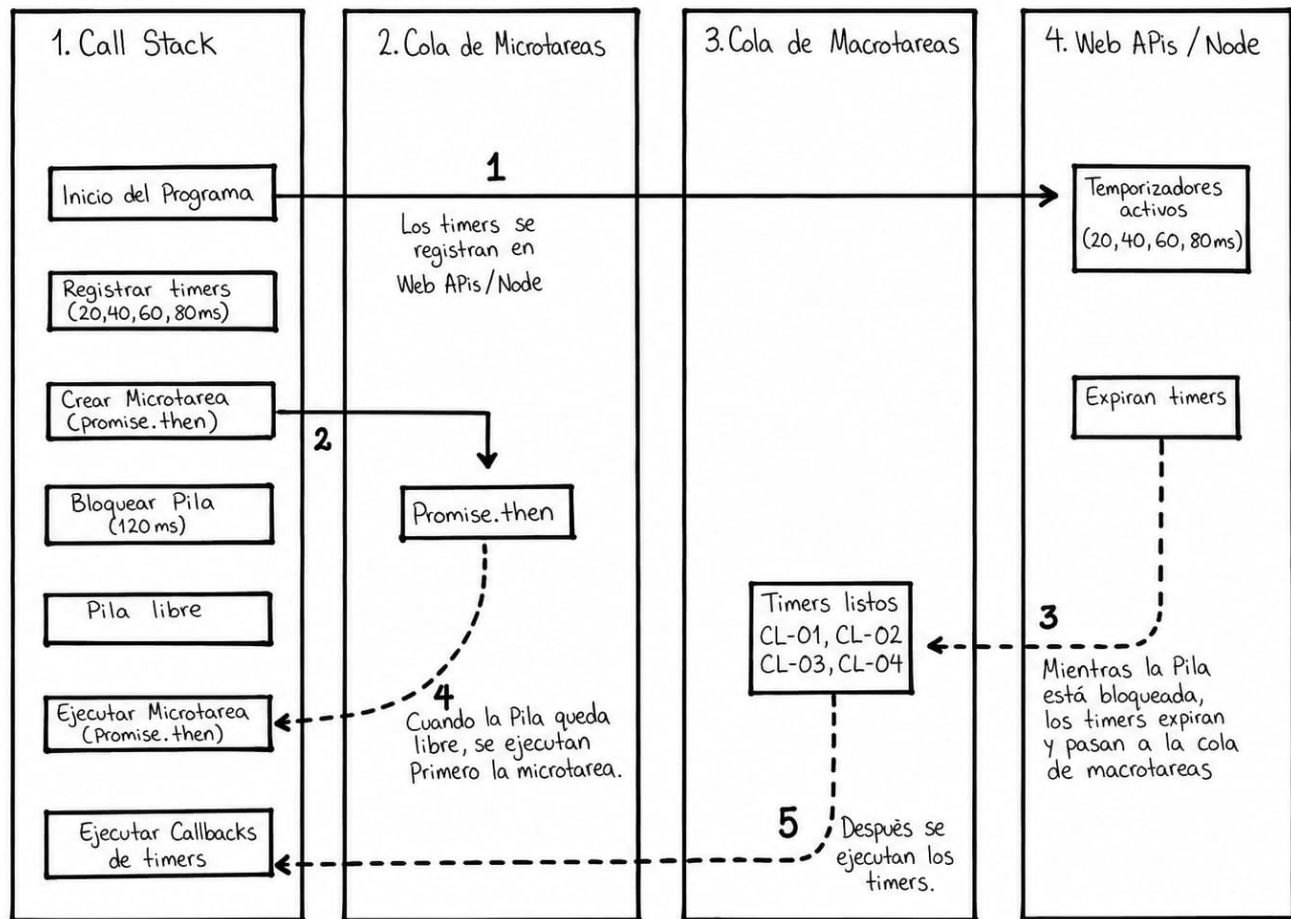
4. Las funciones de temporización en JavaScript no garantizan tiempos de ejecución, sino tiempos mínimos de espera.

Link repositorio GitHub <https://github.com/Nessaart/JavaScriptAvanzado-NivelBasico.git>

REFERENCIAS

- [1] Node.js Docs, "The Node.js Event Loop, Timers, and process.nextTick()," 2023. [En línea]. Disponible en: <https://nodejs.org/en/docs/guides/event-loop-timers-and-nexttick>
- [2] a Guía Definitiva del Event Loop en JS (async, promises, callbacks, render) [En línea]. Disponible en: <https://www.youtube.com/watch?v=dX2lThXc0p4&t=3s>

VI. DIAGRAMA DEL MODELO DE EJECUCIÓN (CASO LÍMITE)



VII. CONCLUSIONES

1. Async es la mejor opción en cuanto a tiempo de espera y orden, por ello es recomendable ya que el control de errores es mucho más manejable comparado con nuestras otras opciones.
2. El bloqueo síncrono sobre el Call Stack afecta el tiempo de respuesta de las tareas asíncronas pendientes.
3. Aunque los timers ya hayan terminado su tiempo de espera, NO pueden ejecutar sus callbacks mientras la pila de ejecución siga ocupada.